

Based on the methodology and evaluation of your low-cost Pedestrian Dead Reckoning (PDR) paper, here are the main technical limitations and drawbacks you have **overcome compared to existing indoor positioning systems**:

1. Zero Infrastructure Dependencies (vs. UWB, Wi-Fi, BLE Beacons)

- **The Problem in Other Systems:** Ultra-Wideband (UWB), Wi-Fi fingerprinting, and Bluetooth Low Energy (BLE) beacon networks require expensive external hardware anchors, active power sources, and labor-intensive pre-deployment mapping/calibration before they can be used inside a building.
- **What You Overcame:** Your system is **100% self-contained and autonomous**. It operates anywhere instantly without requiring pre-installed indoor beacons or external signal infrastructure.

2. Unaffected by GNSS Indoor Blackouts (vs. GPS / GNSS)

- **The Problem in Other Systems:** Global Navigation Satellite Systems (GNSS / GPS) completely fail inside multi-story concrete or metal buildings due to signal attenuation, multi-path reflections, and line-of-sight blockage.
- **What You Overcame:** By relying entirely on onboard MEMS sensor fusion (accelerometer + magnetometer), your system works reliably in **GNSS-denied environments** like basements, tunnels, and deep indoor corridors.

3. Stopped Exponential Distance Drift (vs. Double Integration $\iint a \, dt$)

- **The Problem in Other Systems:** Naïve IMU tracking attempts to calculate distance by directly integrating raw acceleration twice ($\iint a \, dt$). Because sensor noise accumulates cubically over time ($O(t^3)$), even high-end sensors drift by dozens of meters within a few seconds.
- **What You Overcame:** You eliminated double integration drift entirely by implementing a **stride-based step-counting approach** ($D = N_{\text{steps}} \times L_{\text{stride}}$). Distance scales linearly with physical footsteps rather than accumulating time-based drift.

4. Rejection of False Steps & Vibration Noise (vs. Basic Threshold Pedometers)

- **The Problem in Other Systems:** Traditional threshold-based pedometers get easily tricked by hand fidgeting, cable tugs, mechanical recoil ("fro" motion), or table vibrations, leading to massive step overcounting.
- **What You Overcame:** You integrated a **Refractory Lockout Window (400 ms)** combined with an **AI Decision Tree State Classifier** (gating via Magnitude & Variance) and an Exponential Moving Average (EMA) low-pass filter. This explicitly filters out non-walking movements, finger taps, and continuous shaking.

5. Cost & Computational Efficiency (vs. High-End Industrial IMUs)

- **The Problem in Other Systems:** High-precision Fiber Optic Gyroscope (FOG) or Ring Laser Gyroscope (RLG) systems are bulky, cost thousands of dollars, and require heavy processors.
- **What You Overcame:** You achieved high positioning accuracy (**distance error under 5%** and **heading drift within $\pm 2^\circ$ to $\pm 5^\circ$**) using a **low-cost MEMS sensor suite** running on a lightweight, resource-constrained microcontroller in MicroPython.

Summary Table for Quick Reference

Feature / Challenge	Traditional / Other Systems	Your Proposed System
Indoor Infrastructure	Requires expensive BLE/UWB beacons or Wi-Fi maps	None (100% autonomous)
GPS Attenuation	Fails completely indoors	Unbuffered indoor operation
Distance Error Rate	Cubic drift ($O(t^3)$) via double integration	Bounded step error (< 5%)
False Step Noise	Tricked by taps, shakes, and cable wiggles	Blocked via AI Classifier + 400ms refractory gate
Hardware Cost	High-end IMUs (\$100s–\$1000s)	Low-cost MEMS + MCU setup

Here is the complete breakdown of how **Accelerometers** and **Magnetometers** work—from absolute physical principles to silicon chip mechanics and raw binary data delivery.

Part 1: The Accelerometer (Detecting Acceleration & Movement)

0. The Fundamentals: What is Acceleration?

Newton's Second Law states:

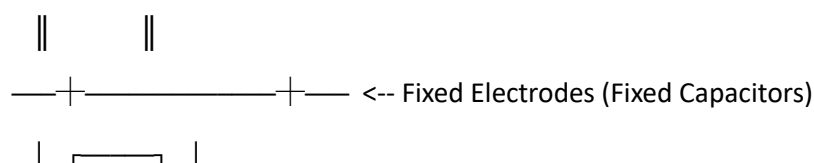
$$\mathbf{F} = m \cdot \mathbf{a}$$

An accelerometer does not directly measure velocity or displacement; it measures **inertial force** exerted on a mass when it speeds up, slows down, or changes direction. It also constantly measures **Earth's gravitational pull** ($1g \approx 9.81 \text{ m/s}^2$).

1. The Microscopic Hardware Engine: MEMS Architecture

Inside a modern chip, the accelerometer is a **MEMS (Micro-Electro-Mechanical Systems)** device. It is literally a microscopic mechanical machine etched out of a single piece of silicon wafer.

[Fixed Silicon Frame]



Springs  Mass  <-- Microscopic Moving Mass (Proof Mass)

  <-- Fixed Electrodes

1. **Proof Mass & Springs:** At the center sits a tiny silicon weight (proof mass) suspended by microscopic silicon springs.
2. **Comb Fingers (Capacitors):** Extending from the proof mass are thousands of interlocking silicon "fingers". Opposite them are fixed silicon fingers attached to the stationary frame of the chip.
3. **The Air Gap:** The tiny gap between the moving fingers and fixed fingers forms a set of microscopic electrical **capacitors**.

2. Physical Detection Mechanism (Movement to Electricity)

1. Resting State:

- When stationary, the springs hold the proof mass centered. The capacitance (C) on both sides is balanced.

2. Motion / Acceleration Occurs:

- When you push or move the sensor, the chip frame moves instantly. However, **inertia** causes the proof mass to lag behind, compressing the microscopic springs.
- This shifts the moving fingers closer to one set of fixed fingers and farther from the other.

3. Capacitance Shift:

- Capacitance changes based on distance (d):

$$C = \frac{\epsilon \cdot A}{d}$$

- The onboard electronics detect this minuscule shift in electrical capacitance (ΔC).

3. Chip Electronics & Signal Flow

Physical Motion —► Mass Shifts —► Capacitance Changes (ΔC)

|



Analog Voltage Signal

|



Analog-to-Digital Converter (ADC)

|



12/16-Bit Binary Output Over I²C

- **Differential Amplifier:** Measures the electrical difference between capacitor plates and outputs a small analog voltage proportional to acceleration.
- **Analog-to-Digital Converter (ADC):** Converts that voltage into a signed 12-bit or 16-bit binary integer.
- **I²C / SPI Bus:** Sends digital integer values (X, Y, Z) to your microcontroller at 100 Hz (100 samples per second).

Part 2: The Magnetometer (Detecting Direction & Heading)

0. The Fundamentals: What does a Magnetometer Measure?

Earth acts like a giant bar magnet with magnetic field lines flowing from south to north. A magnetometer measures the strength and direction of local **ambient magnetic fields** (measured in microteslas, μT , or gauss, G).

1. The Physical Mechanism: Anisotropic Magnetoresistance (AMR)

Most modern MEMS magnetometers use **Anisotropic Magnetoresistive (AMR)** silicon sensors.

Unbiased Magnetic Domains

[↗ ↘ ↖ ↙ → ↠]



| External Magnetic Field (Earth's North)



Aligned Magnetic Domains

[↑ ↑ ↑ ↑ ↑ ↑]

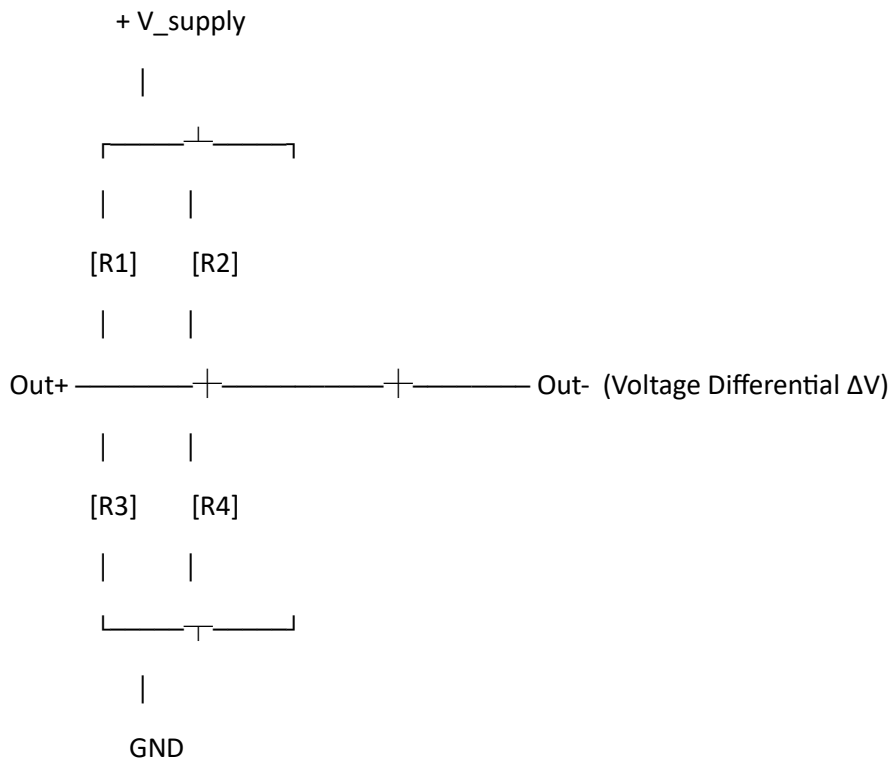


Electrical Resistance Shifts (ΔR)

1. **Thin-Film Permalloy Strips:** The chip contains tiny strips of Permalloy (a nickel-iron magnetic alloy) deposited onto silicon.
2. **Domain Alignment:** When an external magnetic field (like Earth's magnetic field) crosses the Permalloy strip, it changes the internal magnetic orientation of the atoms.
3. **Resistance Change:** This realignment changes the material's **electrical resistance (R)** depending on the angle (θ) between the electric current and the magnetic field vector.

2. Wheatstone Bridge Circuit Architecture

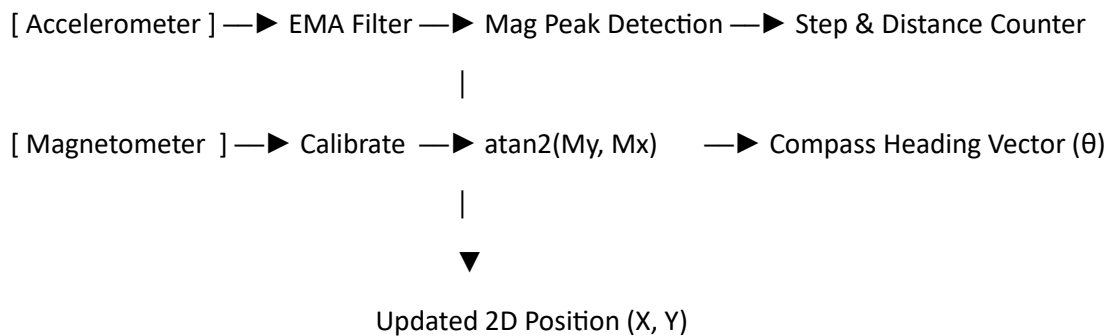
To detect tiny resistance changes, the Permalloy strips are arranged in a **Wheatstone Bridge** configuration for each axis (X, Y, Z):



- When the sensor aligns with Magnetic North, the resistance in opposite arms shifts, causing a differential voltage change (ΔV) across Out+ and Out- .
- The onboard ADC digitizes this differential voltage into magnetic flux density readings (M_x, M_y, M_z).

Part 3: Sensor Fusion (Putting It Together for Pedestrian Dead Reckoning)

When you combine both sensors inside a navigation system, they work in harmony to calculate distance and compass heading:



1. **Accelerometer Function:** Measures total dynamic acceleration magnitude ($\Delta M = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$). The algorithm runs filtering to identify physical footfall impact peaks and counts steps ($D = N_{\text{steps}} \times L_{\text{stride}}$).
2. **Magnetometer Function:** Measures the horizontal magnetic vector to compute the angle relative to magnetic North:

$$\theta = \text{atan2}(M_y - \text{bias}_y, M_x - \text{bias}_x) \times \left(\frac{180}{\pi}\right)$$

3. **Position Integration:** Updates 2D spatial position on each detected step:

$$X_{\text{new}} = X_{\text{old}} + L_{\text{stride}} \cdot \sin(\theta)$$

$$Y_{\text{new}} = Y_{\text{old}} + L_{\text{stride}} \cdot \cos(\theta)$$

Here is the complete breakdown of how **Accelerometers** and **Magnetometers** work—from absolute physical principles to silicon chip mechanics and raw binary data delivery.

Part 1: The Accelerometer (Detecting Acceleration & Movement)

0. The Fundamentals: What is Acceleration?

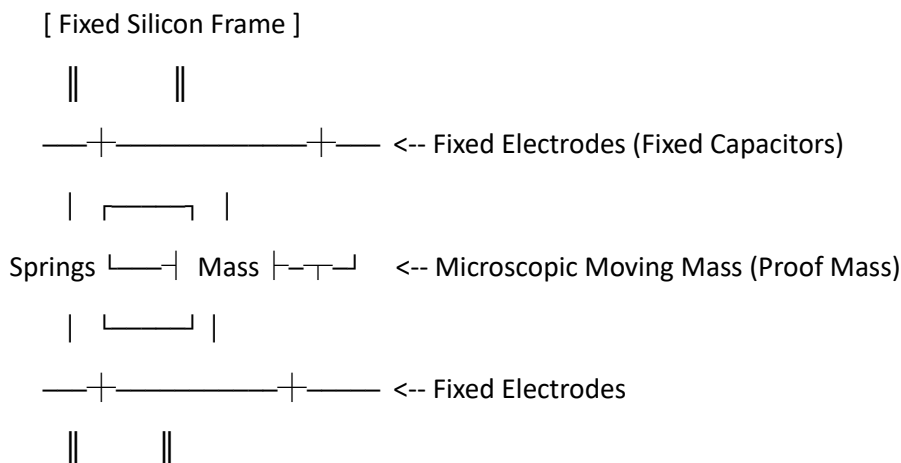
Newton's Second Law states:

$$\mathbf{F} = m \cdot \mathbf{a}$$

An accelerometer does not directly measure velocity or displacement; it measures **inertial force** exerted on a mass when it speeds up, slows down, or changes direction. It also constantly measures **Earth's gravitational pull** ($1g \approx 9.81 \text{ m/s}^2$).

1. The Microscopic Hardware Engine: MEMS Architecture

Inside a modern chip, the accelerometer is a **MEMS (Micro-Electro-Mechanical Systems)** device. It is literally a microscopic mechanical machine etched out of a single piece of silicon wafer.



1. **Proof Mass & Springs:** At the center sits a tiny silicon weight (proof mass) suspended by microscopic silicon springs.
2. **Comb Fingers (Capacitors):** Extending from the proof mass are thousands of interlocking silicon "fingers". Opposite them are fixed silicon fingers attached to the stationary frame of the chip.
3. **The Air Gap:** The tiny gap between the moving fingers and fixed fingers forms a set of microscopic electrical **capacitors**.

2. Physical Detection Mechanism (Movement to Electricity)

1. Resting State:

- When stationary, the springs hold the proof mass centered. The capacitance (C) on both sides is balanced.

2. Motion / Acceleration Occurs:

- When you push or move the sensor, the chip frame moves instantly. However, **inertia** causes the proof mass to lag behind, compressing the microscopic springs.
- This shifts the moving fingers closer to one set of fixed fingers and farther from the other.

3. Capacitance Shift:

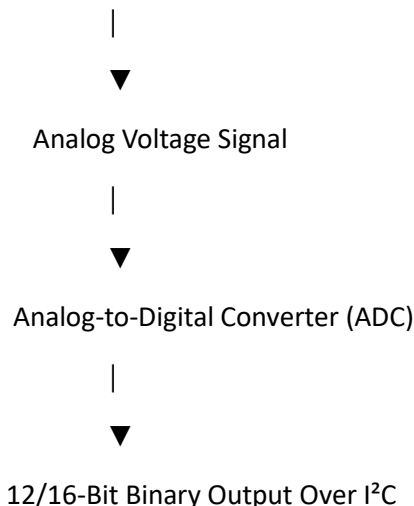
- Capacitance changes based on distance (d):

$$C = \frac{\epsilon \cdot A}{d}$$

- The onboard electronics detect this minuscule shift in electrical capacitance (ΔC).

3. Chip Electronics & Signal Flow

Physical Motion —► Mass Shifts —► Capacitance Changes (ΔC)



- **Differential Amplifier:** Measures the electrical difference between capacitor plates and outputs a small analog voltage proportional to acceleration.
- **Analog-to-Digital Converter (ADC):** Converts that voltage into a signed 12-bit or 16-bit binary integer.
- **I²C / SPI Bus:** Sends digital integer values (X, Y, Z) to your microcontroller at 100 Hz (100 samples per second).

Part 2: The Magnetometer (Detecting Direction & Heading)

0. The Fundamentals: What does a Magnetometer Measure?

Earth acts like a giant bar magnet with magnetic field lines flowing from south to north. A magnetometer measures the strength and direction of local **ambient magnetic fields** (measured in microteslas, μT , or gauss, G).

1. The Physical Mechanism: Anisotropic Magnetoresistance (AMR)

Most modern MEMS magnetometers use **Anisotropic Magnetoresistive (AMR)** silicon sensors.

Unbiased Magnetic Domains

[↗ ↘ ↖ ↙ → ↠]

|

| External Magnetic Field (Earth's North)



Aligned Magnetic Domains

[↑ ↑ ↑ ↑ ↑ ↑]

|

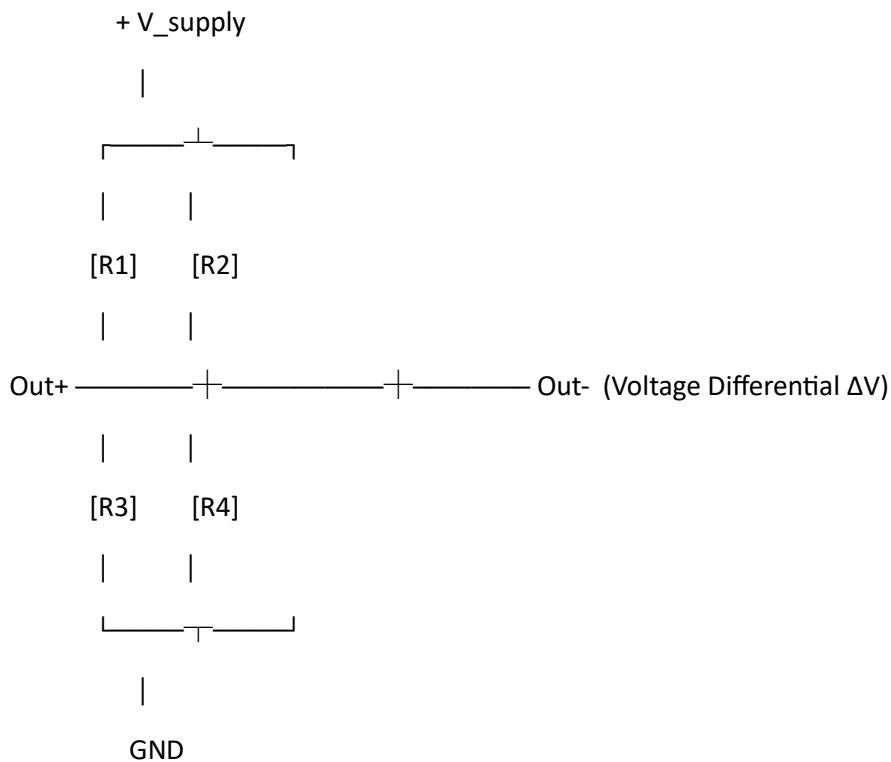


Electrical Resistance Shifts (ΔR)

1. **Thin-Film Permalloy Strips:** The chip contains tiny strips of Permalloy (a nickel-iron magnetic alloy) deposited onto silicon.
2. **Domain Alignment:** When an external magnetic field (like Earth's magnetic field) crosses the Permalloy strip, it changes the internal magnetic orientation of the atoms.
3. **Resistance Change:** This realignment changes the material's **electrical resistance (R)** depending on the angle (θ) between the electric current and the magnetic field vector.

2. Wheatstone Bridge Circuit Architecture

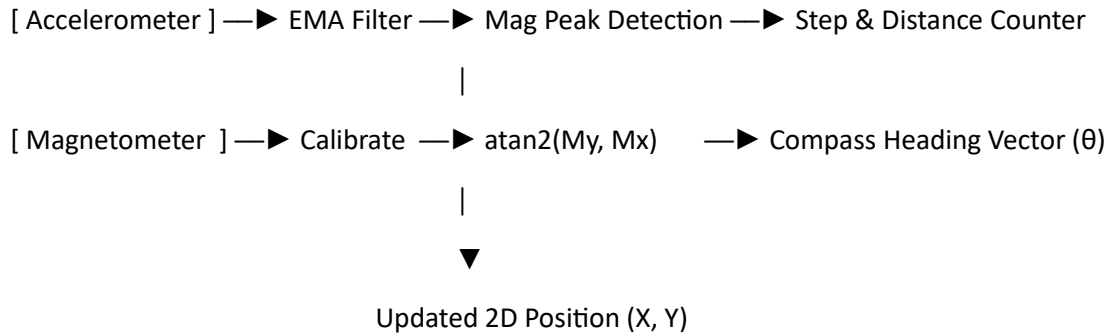
To detect tiny resistance changes, the Permalloy strips are arranged in a **Wheatstone Bridge** configuration for each axis (X, Y, Z):



- When the sensor aligns with Magnetic North, the resistance in opposite arms shifts, causing a differential voltage change (ΔV) across Out+ and Out-.
- The onboard ADC digitizes this differential voltage into magnetic flux density readings (M_x, M_y, M_z).

Part 3: Sensor Fusion (Putting It Together for Pedestrian Dead Reckoning)

When you combine both sensors inside a navigation system, they work in harmony to calculate distance and compass heading:



1. **Accelerometer Function:** Measures total dynamic acceleration magnitude ($\Delta M = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$). The algorithm runs filtering to identify physical footfall impact peaks and counts steps ($D = N_{\text{steps}} \times L_{\text{stride}}$).
2. **Magnetometer Function:** Measures the horizontal magnetic vector to compute the angle relative to magnetic North:

$$\theta = \text{atan2}(M_y - \text{bias}_y, M_x - \text{bias}_x) \times \left(\frac{180}{\pi}\right)$$

3. **Position Integration:** Updates 2D spatial position on each detected step:

$$X_{\text{new}} = X_{\text{old}} + L_{\text{stride}} \cdot \sin(\theta)$$

$$Y_{\text{new}} = Y_{\text{old}} + L_{\text{stride}} \cdot \cos(\theta)$$

Here is the complete **0-to-Hero technical journey** of what happens when you physically pick up, slide, or move the sensor in your hand or on a desk—from physical kinetic energy down to registers, buses, and code calculations!

Stage 1: The Physical Motion (Kinetic Energy & Inertia)

Imagine holding the sensor board on your desk. When you move it forward:

1. **At Rest:** Everything inside the chip is stationary. The only force acting on the accelerometer is Earth's gravity ($1g \approx 9.81 \text{ m/s}^2$) pulling downward along the Z-axis.
2. **The Acceleration Phase (The Push):** When your hand pushes the board forward along the desk (X-axis), the **chip housing moves immediately**.

3. **The Inertial Lag:** Inside the microscopic MEMS chamber, the suspended silicon mass wants to stay at rest due to **Newton's First Law of Motion (Inertia)**. As a result, the silicon mass lags behind and bends its microscopic springs backward relative to the chip frame.
4. **The Deceleration Phase (The Stop):** When your hand stops the board at the end of the slide, the chip housing stops, but the silicon mass keeps surging forward, bending the springs in the opposite direction.

Stage 2: Mechanical to Electrical Transduction (Capacitance & Voltage)

How does a bent silicon spring turn into electricity?

Plaintext

[Physical Movement]



[Mass Lags Behind (Inertia)]



[Microscopic Capacitor Gap Changes (Δd)]



[Capacitance Shifts (ΔC)]



[Differential Charge / Voltage Created (ΔV)]

1. **Capacitor Electrodes:** Attached to the moving silicon mass are thousands of microscopic fingers interlocked with stationary fingers on the chip frame.
2. **Capacitance Equation:** The capacitance (C) between two microscopic fingers depends on the distance (d) separating them:

$$C = \frac{\epsilon \cdot A}{d}$$

3. **The Shift:** As the mass bends the springs by just a few nanometers, d shrinks on one side and expands on the other.
4. **Current Generation:** This physical change in gap distance forces electrical charge to flow, creating a tiny differential voltage (ΔV) proportional to the exact force of your hand movement.

Stage 3: Inside the ASIC (Analog to Digital Conversion)

Right next to the microscopic mechanical silicon structure inside the LSM303 chip sits an **ASIC (Application-Specific Integrated Circuit)** chip.

Plaintext

Analog Voltage (ΔV)



[Charge Amplifier] —► Boosts tiny microvolt signals



[12-Bit ADC Engine] —► Samples analog voltage at 100 Hz



[Output Registers] —► Stores signed 16-bit 2's Complement Binary

1. **Amplification:** The ASIC boosts the minuscule voltage generated by the capacitor plates.
2. **Sampling (100 Hz):** 100 times every second (every 10 milliseconds), the internal **Analog-to-Digital Converter (ADC)** snaps a snapshot of the voltage levels for *X*, *Y*, and *Z* axes.
3. **Conversion to Digital Counts:** The ADC converts the voltage into raw 16-bit integers (called "LSB counts"):
 - 0 LSB = Zero acceleration ($0g$).
 - +1024 LSB = Positive $+1g$ acceleration.
 - −1024 LSB = Negative $-1g$ acceleration.
4. **Register Storage:** These integer values are written into specific hardware memory registers inside the chip:
 - OUT_X_L_A & OUT_X_H_A (X-axis Low/High Byte)
 - OUT_Y_L_A & OUT_Y_H_A (Y-axis Low/High Byte)
 - OUT_Z_L_A & OUT_Z_H_A (Z-axis Low/High Byte)

Stage 4: Data Transfer over I^2C Bus to Microcontroller

Now the raw motion numbers are trapped inside the sensor chip's memory registers. How does your Python code get them?

Plaintext

[Microcontroller] — SDA (Data Line) —► [Sensor Chip]

◄— SCL (Clock Line) —

1. **The Request:** Your microcontroller acts as the I²C Master. Every 10 ms, it sends a command over the **SDA (Serial Data)** line:

"Hey sensor at address 0x19, give me 6 bytes starting from register OUT_X_L_A!"

2. **The Transmission:** The sensor chip responds by clocking out 6 bytes of binary data over the SDA wire at 400 kHz.
3. **Byte Assembly:** MicroPython receives the high byte and low byte for each axis and bit-shifts them together into full integer values:

Python

```
raw_x = (high_byte << 8) | low_byte
```

Stage 5: Software Processing & Decision Execution

Once MicroPython receives raw_x, raw_y, and raw_z, it executes the algorithm pipeline:

Plaintext

Raw (X, Y, Z) Integers

|



[1. Remove Static Calibration Offsets]

|



[2. EMA Filter (Smooth Jitter)] —► $S_t = \alpha * X_t + (1 - \alpha) * S_{(t-1)}$

|



[3. Calculate 3D Vector Magnitude] —► $\Delta M = \sqrt{\Delta X^2 + \Delta Y^2 + \Delta Z^2}$

|



[4. Compute Rolling Variance] —► Checks motion stability over 10 samples

|



[5. AI Classifier + Cooldown Gate]

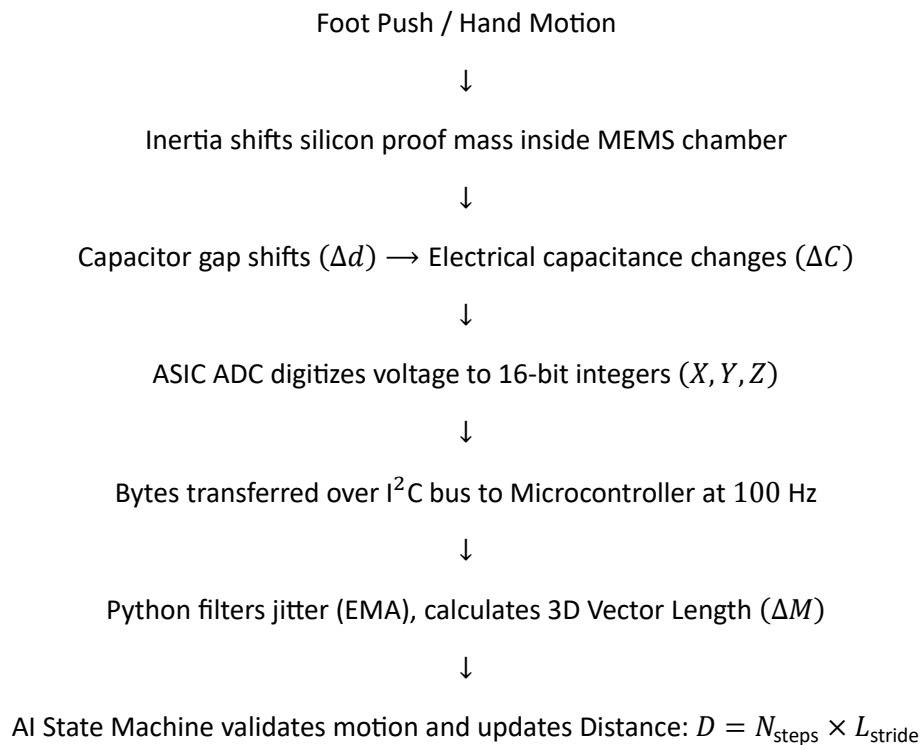
|

└─► If Magnitude > Ceiling —► "TAP_NOISE" (Ignore)

└─► If Variance > Threshold —► "SHAKING" (Freeze)

└─► If Valid Walk Peak —► "WALKING" —► STEP + 1 & DISTANCE + Stride

Summary Flow (From Footstep to Terminal Display)



Here is the **0-to-Hero deep dive** into how physical analog movement gets converted into digital binary bits, assembled into 16-bit integers, transferred over wires, and processed by your code.

1. What is Bit Conversion?

At its core, bit conversion is taking a **continuous physical value** (like voltage from a bent silicon spring) and converting it into a **discrete binary number** (0s and 1s) that a computer processor can understand.

Inside the sensor, this is handled by an **Analog-to-Digital Converter (ADC)**.

2. Understanding ADC Resolution & Full-Scale Range

To understand the numbers, you need to know two key concepts:

- Resolution (Bits):** Most MEMS sensors (like the LSM303 or MPU6050) use a **16-bit ADC**.
 - A 16-bit register can hold $2^{16} = 65,536$ unique values.
 - Since acceleration can be positive (+) or negative (−), the sensor uses **Signed 2's Complement** arithmetic:
 - Range: $-32,768$ to $+32,767$ LSB (Least Significant Bits).
- Full-Scale Range (Sensitivity Setting):** You can program the sensor's measurement range (e.g., $\pm 2g$, $\pm 4g$, $\pm 8g$, or $\pm 16g$).
 - If set to $\pm 2g$, the total span is $4g$ (from $-2g$ to $+2g$).
 - Sensitivity Scale Factor:**

$$\text{Scale Factor} = \frac{32,768 \text{ LSB}}{2g} = 16,384 \text{ LSB/g}$$

- That means **1g of force generates an integer count of 16,384**.

3. Step-by-Step Example: Moving the Sensor Forward

Let's walk through a real example where you push the sensor forward along the *X*-axis, creating a force of +0.5g.

Step A: Analog Voltage Generation

1. The physical force pushes the MEMS mass, shifting capacitance (ΔC).
2. The amplifier converts this into an analog voltage:
 - 0 V bias point = 0g
 - +0.5 V generated = +0.5g acceleration

Step B: The ADC Conversion

The 16-bit ADC measures the +0.5 V signal and converts it to digital counts:

$$\text{Digital Value} = +0.5g \times 16,384 \text{ LSB/g} = +8,192 \text{ LSB}$$

In binary (16-bit signed integer):

$$+8,192_{10} = \mathbf{00100000\ 00000000}_2$$

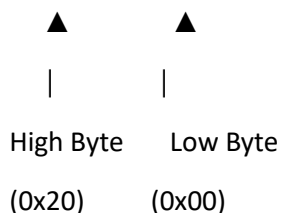
Step C: Splitting into High and Low Bytes

Memory registers in hardware chips are only **8 bits (1 Byte)** wide. Therefore, the 16-bit integer must be split into two 8-bit bytes:

- **High Byte (OUT_X_H_A):** The first 8 bits → **00100000₂** (32₁₀ or 0x20 in Hex)
- **Low Byte (OUT_X_L_A):** The last 8 bits → **00000000₂** (0₁₀ or 0x00 in Hex)

Plaintext

16-bit Number: [0 0 1 0 0 0 0 0 | 0 0 0 0 0 0 0 0]



Step D: Transmission Over I²C Bus

The sensor sends these two bytes over the data wire (SDA) as electrical pulses (high voltage = 1, low voltage = 0):

Plaintext

Low Byte (0x00): _____ (8 low pulses)

High Byte (0x20): `__T_____` (1 high pulse at bit position 5)

Step E: Reassembling the Bits in Python

When your microcontroller receives the two 8-bit bytes, it needs to recombine them into a single 16-bit integer using **Bit-Shifting** and **Bitwise OR (|)**.

Python

```
# Received bytes from I2C
```

```
low_byte = 0x00 # 00000000 in binary
```

```
high_byte = 0x20 # 00100000 in binary
```

```
# 1. Shift High Byte 8 positions to the left
```

```
# 00100000 becomes 00100000 00000000
```

```
shifted_high = high_byte << 8
```

```
# 2. Combine with Low Byte using Bitwise OR (|)
```

```
raw_x = shifted_high | low_byte
```

```
print(raw_x) # Output: 8192
```

Step F: Converting Bits Back to Acceleration (m/s^2 or g)

Now that Python has the reassembled integer (8192), it converts it back into meaningful physical units:

Python

```
# Convert LSB counts to 'g' force
```

```
accel_g = raw_x / 16384.0 # 8192 / 16384 = +0.5g
```

```
# Convert 'g' to meters per second squared ( $\text{m/s}^2$ )
```

```
accel_m_s2 = accel_g * 9.80665 # 0.5 * 9.80665 = +4.903  $\text{m/s}^2$ 
```

What About Negative Numbers? (Negative Acceleration / Braking)

If you pull the sensor backward at $-0.5g$, the ADC outputs **-8,192 LSB**.

Computers represent negative numbers using **2's Complement**:

1. +8,192 in binary: 00100000 00000000
2. Invert all bits (1's complement): 11011111 11111111
3. Add 1 (2's complement): 11100000 00000000 (Hex: 0xE000)

In Python, if the most significant bit (leftmost bit) is 1, the value is negative:

Python

```
if raw_x > 32767:
```

```
    raw_x -= 65536 # Converts unsigned binary to signed negative integer
```

```
print(raw_x) # Output: -8192
```

Summary Matrix

Physical Motion	Voltage (ΔV)	ADC 16-bit Binary	Hex (High/Low)	Raw Integer	Calculated Acceleration
Max Forward (+2g)	+2.0 V	01111111 11111111	0x7F 0xFF	+32,767	+2.00g (+19.62 m/s ²)
Push Forward (+0.5g)	+0.5 V	00100000 00000000	0x20 0x00	+8,192	+0.50g (+4.90 m/s ²)
At Rest (0g)	0.0 V	00000000 00000000	0x00 0x00	0	0.00g (0.00 m/s ²)
Pull Back (-0.5g)	-0.5 V	11100000 00000000	0xE0 0x00	-8,192	-0.50g (-4.90 m/s ²)
Max Backward (-2g)	-2.0 V	10000000 00000000	0x80 0x00	-32,768	-2.00g (-19.62 m/s ²)

Here is the complete **0-to-Hero guide** covering what stride length is, how it works mathematically, and how your system's code and sensor detect whether a person is sitting, walking, moving, or shaking!

Part 1: What is Stride Length? (0-to-Hero)

1. The Core Definition

- **Step Length:** The distance measured from the heel strike of **one foot** to the heel strike of the **other foot** (e.g., Left foot → Right foot). For an average adult, this is roughly 0.65--0.75 meters (65--75 cm).

- **Stride Length:** The distance measured between two consecutive heel strikes of the **same foot** (e.g., Left foot → Right foot → Left foot). One stride equals **two steps** ($L_{\text{stride}} = 2 \times \text{Step Length}$).

Note in Your Project: In Pedestrian Dead Reckoning (PDR) code, the variable STRIDE_LENGTH_M (e.g., set to 0.05 m for tabletop tests or 0.70 m for real walking) represents the distance assigned to **each detected step peak**.

2. How Stride Length Calculates Total Distance

Every time your algorithm detects a valid step peak from the accelerometer, it updates the total displacement using a simple accumulation equation:

$$\text{Distance}_{\text{total}} = N_{\text{steps}} \times L_{\text{stride}}$$

If you turn or walk in 2D space, it uses vector trigonometry with your compass heading (θ):

$$X_{\text{new}} = X_{\text{old}} + L_{\text{stride}} \cdot \sin(\theta)$$

$$Y_{\text{new}} = Y_{\text{old}} + L_{\text{stride}} \cdot \cos(\theta)$$

Part 2: How the Sensor Detects Sitting, Walking, Moving, and Shaking

Your microcontroller doesn't "see" a human body—it only sees a stream of 3D acceleration numbers (a_x, a_y, a_z) coming in 100 times every second (100 Hz).

To classify motion into **Sitting, Walking, Moving, or Shaking**, the system uses two key mathematical features:

1. **Vector Magnitude (ΔM):** How strong the physical force is.
2. **Rolling Variance (σ^2):** How chaotic/rhythmic the signal is over a short time window (e.g., 10 samples).

Step A: Calculating 3D Vector Magnitude (ΔM)

When the sensor moves, it accelerates along X , Y , and Z simultaneously. The total combined force vector is calculated using the 3D Pythagorean theorem:

$$\Delta M = \sqrt{(a_x - \text{bias}_x)^2 + (a_y - \text{bias}_y)^2 + (a_z - \text{bias}_z)^2}$$

- **At Rest (Sitting):** $\Delta M \approx 0 \text{ m/s}^2$ (after gravity/bias subtraction).
- **Walking:** ΔM creates smooth, rhythmic wave spikes between 20--40 m/s^2 .
- **Violent Shaking:** ΔM spikes wildly past 100--200 m/s^2 .

Step B: Calculating Signal Variance (σ^2)

Variance measures how much the acceleration values bounce around over a moving buffer of recent samples:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (\Delta M_i - \mu)^2$$

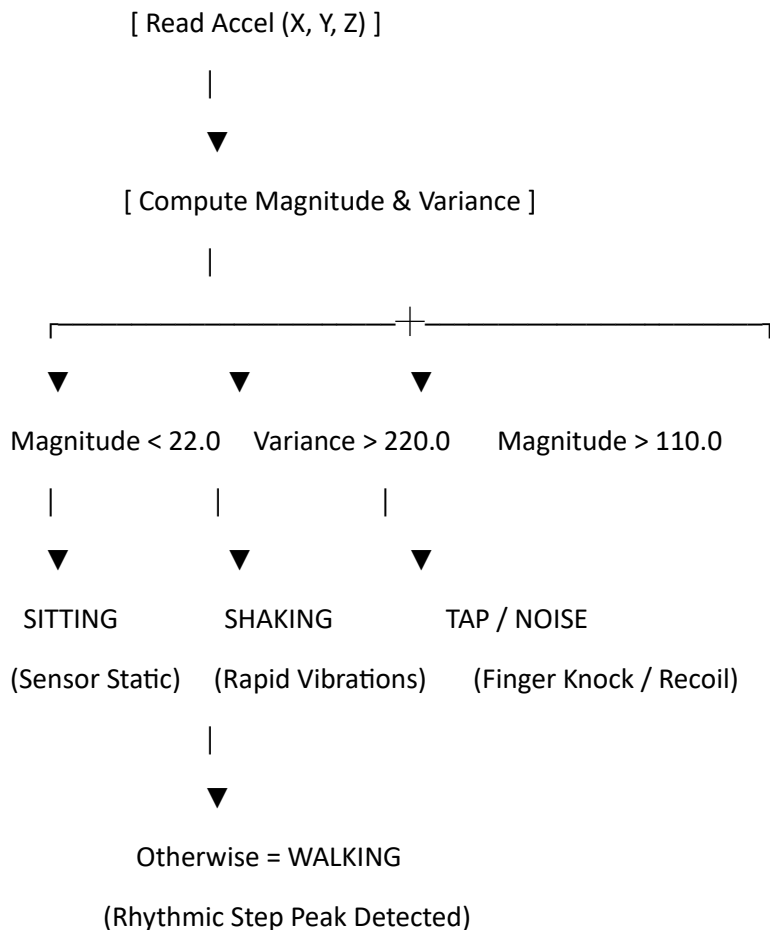
Where μ is the average force over the last 10 readings.

- **Low Variance:** Motion is smooth, constant, or completely static.
- **High Variance:** Motion is erratic, shaky, or rapidly fluctuating.

Part 3: The AI Decision Tree Classifier (How States are Decided)

The Python code feeds the calculated **Magnitude (ΔM)** and **Variance (σ^2)** into a rule-based Decision Tree state machine every 10 ms:

Plaintext



1. Detecting "SITTING" (Static / Idle State)

- **Physics:** The sensor is sitting flat on a desk or held still in a seated person's hand.
- **Condition:** $\Delta M < 22.0$ LSB.
- **What Code Does:** The force magnitude is too weak to be a human footfall. Step counters freeze, and the system logs "SITTING".

2. Detecting "SHAKING" (Erratic / Continuous Noise)

- **Physics:** The sensor is being shaken back and forth rapidly in the air or subjected to vehicle engine vibrations.
- **Condition:** Variance $\sigma^2 > 220.0$.

- **What Code Does:** Real human footsteps have a rhythmic cadence (1--2 steps per second). Shaking produces rapid, continuous, non-rhythmic direction flips. The high variance triggers a safety lock, preventing the step counter from falsely inflating by hundreds of steps.

3. Detecting "TAP / NOISE" (Hand Fidgets & Table Friction)

- **Physics:** You tap your finger on the sensor or slide it across a rough desk, creating a sudden sharp impact spike.
- **Condition:** Magnitude $\Delta M > \text{TAP_SPIKE_CEILING}$ (110.0).
- **What Code Does:** Real walking forces rise and fall smoothly over 200--300 ms. Sudden 1-ms impact spikes are flagged as "TAP_NOISE" and discarded.

4. Detecting True "WALKING" & Counting a Step

- **Physics:** A person takes an intentional step. The heel hits the ground (Heel Strike → Peak Force), the body shifts weight, and the foot lifts (Pause).
- **Conditions Required:**
 1. State classified as WALKING ($\Delta M \geq 22.0$ and Variance ≤ 220.0).
 2. Acceleration magnitude passes the peak threshold.
 3. **Refractory Cooldown Gate:** At least 400 ms (STEP_COOLDOWN_MS) has passed since the last counted step.

Python

Code logic executing inside main loop:

```
now = utime.ticks_ms()
```

```
if activity == "WALKING" and filtered_mag > 22.0:
```

```
    # Cooldown gate blocks double-counting during a single step swing
```

```
    if utime.ticks_diff(now, last_step_time) > 400:
```

```
        step_count += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        last_step_time = now
```

Summary Matrix

State	Acceleration Magnitude (ΔM)	Signal Variance (σ^2)	Code Action
SITTING	Very Low (< 22.0)	Near Zero	Ignore; Distance unchanged

State	Acceleration Magnitude (ΔM)	Signal Variance (σ^2)	Code Action
MOVING / SLIDING	Moderate (> 22.0)	Low / Smooth	Evaluated for step peak validation
WALKING	Peak (22.0--110.0)	Moderate (Rhythmic)	Step Count +1, Distance + L_{stride}
SHAKING	Fluctuating High	High (> 220.0)	Lockout triggered; Steps blocked
TAP / RECOIL	Extreme Spike (> 110.0)	Spiked	Discarded as transient noise